

Sprawozdanie z sieci Kohonena

Imię i nazwisko: [Piotr Wysocki]

Numer indeksu: [333171]

3 maja 2026

1 Krótki opis tematu laboratoriów

Celem laboratorium była implementacja Samoorganizującej się Mapy (sieci Kohonena) z elastyczną konfiguracją hiperparametrów (m.in. topologii siatki, funkcji sąsiedztwa oraz wygaszania współczynnika uczenia).

Działanie algorytmu przetestowano w dwóch etapach:

- **Na danych syntetycznych** (2D sześciokąt, 3D sześcian) – do wizualnej weryfikacji procesu układania się siatki neuronów.
- **Na danych rzeczywistych** (MNIST oraz zbiór treningowy aktywności ze smartfonów) – do klasteryzacji złożonych, wielowymiarowych problemów.

Ostateczną jakość klasteryzacji dla danych rzeczywistych ewaluowano poprzez porównanie wyników z oryginalnymi etykietami. Oceniono zgodność liczby wykrytych klastrów z faktyczną liczbą klas, czystość neuronów oraz ich końcowe rozmieszczenie w przestrzeni cech.

2 Zakres użycia dużych modeli językowych (LLM)

Podczas przygotowywania niniejszego sprawozdania oraz implementacji kodu, duże modele językowe były wykorzystywane w następującym zakresie:

- **Generowanie kodu i planowanie architektury rozwiązania:** wsparcie w tworzeniu fragmentów algorytmów oraz implementacji wybranych elementów rozwiązania.
- **Tłumaczenie pojęć:** pomoc w zrozumieniu, definiowaniu i wyjaśnianiu złożonych zagadnień teoretycznych.
- **Wyszukiwanie zależności:** wsparcie w analizie powiązań między hiperparametrami, danymi i wynikami.
- **Sugerowanie możliwych popraw rozwiązań:** optymalizacja implementacji, identyfikacja błędów oraz poszukiwanie alternatywnych, skuteczniejszych podejść do problemu.
- **Generowanie kodu w LaTeX:** tworzenie struktury dokumentu, szablonów sprawozdań oraz formatowanie list i tabel.
- **Formatowanie zdań w sprawozdaniu:** korekta gramatyczna i stylistyczna, poprawa płynności i czytelności tekstu.

3 Syntetyczne zbiory danych

Dla wszystkich eksperymentów w tej części ustalono ziarno losowe na 0.

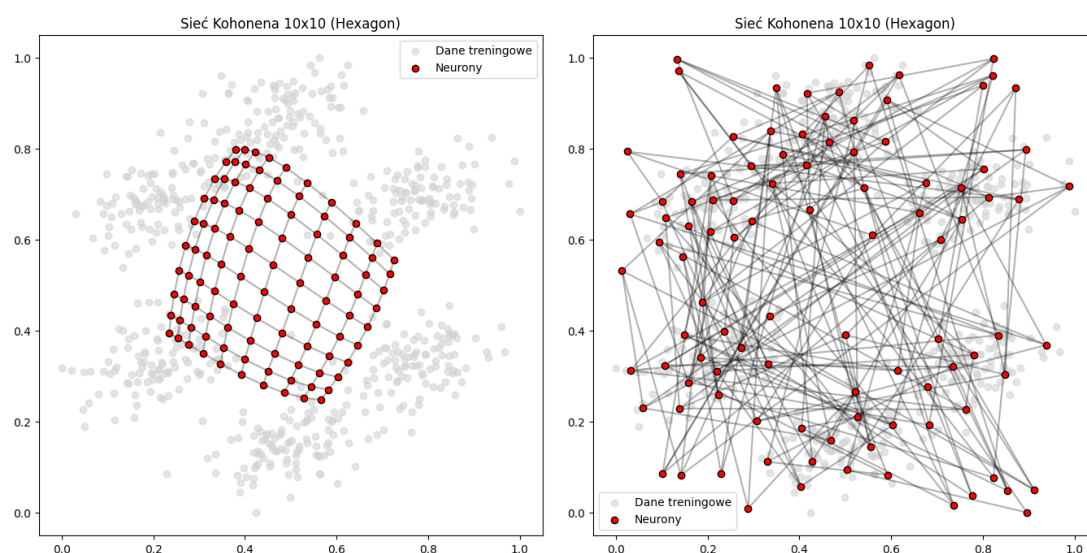
3.1 Zbiór danych Hexagon

W tej części eksperymentów (dotyczących syntetycznych zbiorów danych) wykorzystano wyłącznie kwadratową architekturę danych. Dla zbioru reprezentującego sześciokąt na płaszczyźnie, składającego się z sześciu klastrów zlokalizowanych w jego wierzchołkach, zastosowano siatkę neuronów o rozmiarze 10x10. Proces uczenia trwał 100 epok, a do wyznaczania wpływu na sąsiednie neurony użyto gaussowskiej funkcji sąsiedztwa.

Głównym celem tego etapu było zbadanie wpływu początkowej szerokości sąsiedztwa na proces formowania się mapy oraz ocenienie jakości klasteryzacji. Testowano następujące wartości tego parametru: 0.2, 0.8, 1, 2 oraz 5.

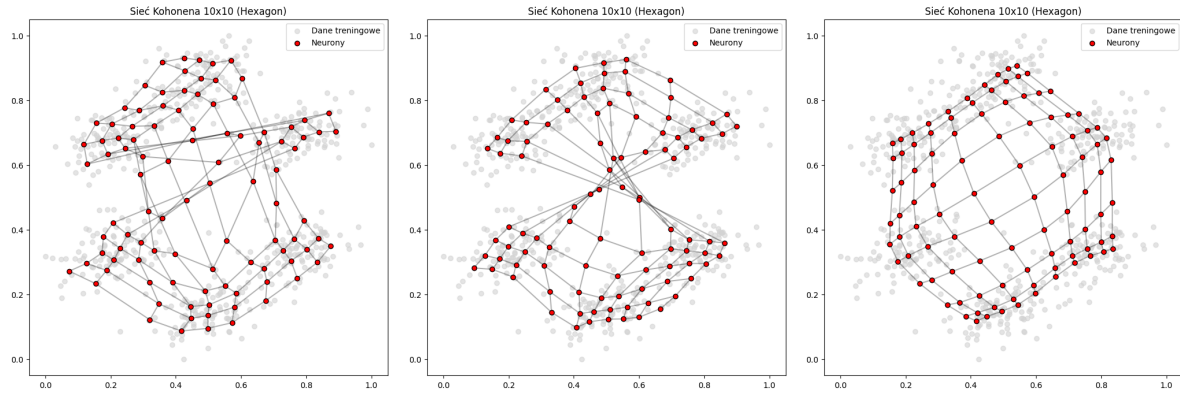
Zaobserwowano, że dla mniejszych wartości sieć zachowywała się w sposób bardziej chaotyczny, skupiając się na lokalnych detalach, podczas gdy przy większych wartościach neurony wykazywały mniejszą tendencję do przemieszczania się. Skrajne wartości parametru przyniosły najgorsze rezultaty dopasowania:

- **Szerokość sąsiedztwa 5:** doprowadziła do *underfittingu* (zbyt słabe dopasowanie do struktury danych). Prawdopodobnie dla tak dużej wartości wymagana byłaby znacznie większa liczba iteracji.
- **Szerokość sąsiedztwa 0.2:** skutkowała *overfittingiem*, tworząc nadmiernie skomplikowaną i "poplątaną" strukturę sieci.



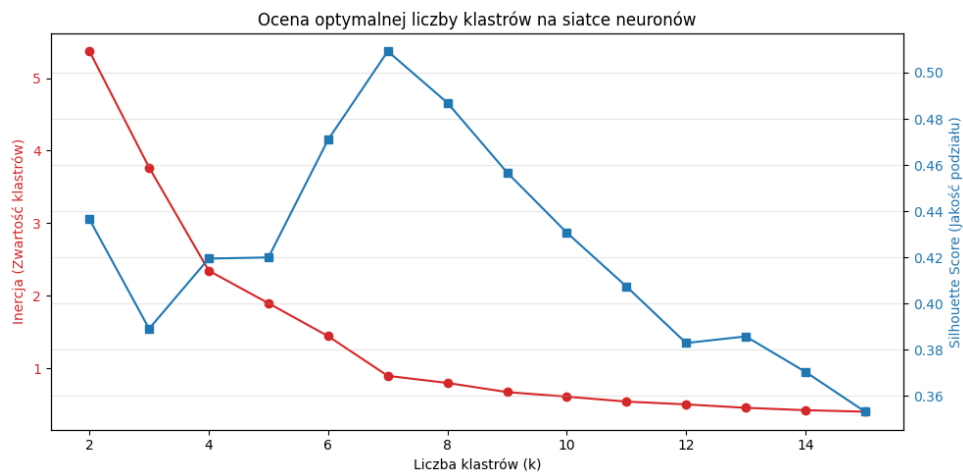
Rysunek 1: Wpływ skrajnych szerokości sąsiedztwa na kształt sieci: underfitting (lewo) z szerokością sąsiedztwa 5 i overfitting (prawo) z szerokością sąsiedztwa 0.2.

Z kolei wartości pośrednie, czyli 0.8, 1 oraz 2, pozwoliły na poprawne i stabilne ułożenie się siatki na zbiorze danych, unikając negatywnych zjawisk niedopasowania lub nadmiernego skomplikowania. Ostatecznie to domyślna wartość promienia równa 1 zapewniła najlepsze, najbardziej równomierne rezultaty wizualne.



Rysunek 2: Poprawne dopasowanie sieci dla pośrednich wartości szerokości sąsiedztwa. Od lewej: 0.8, 1 oraz 2.

Dla uzyskanych przy optymalnym parametrze 1 wag sieci neuronowej, przeprowadzono dodatkową klasteryzację za pomocą algorytmu KMeans, testując liczbę klastrow (k) od 2 do 15. Najwyższą wartość miary *Silhouette Score* oraz wyraźne przezięcie na wykresie osypiska (tzw. metoda łokcia) zaobserwowano dla $k = 6$, co idealnie pokrywa się z faktyczną strukturą zbioru.



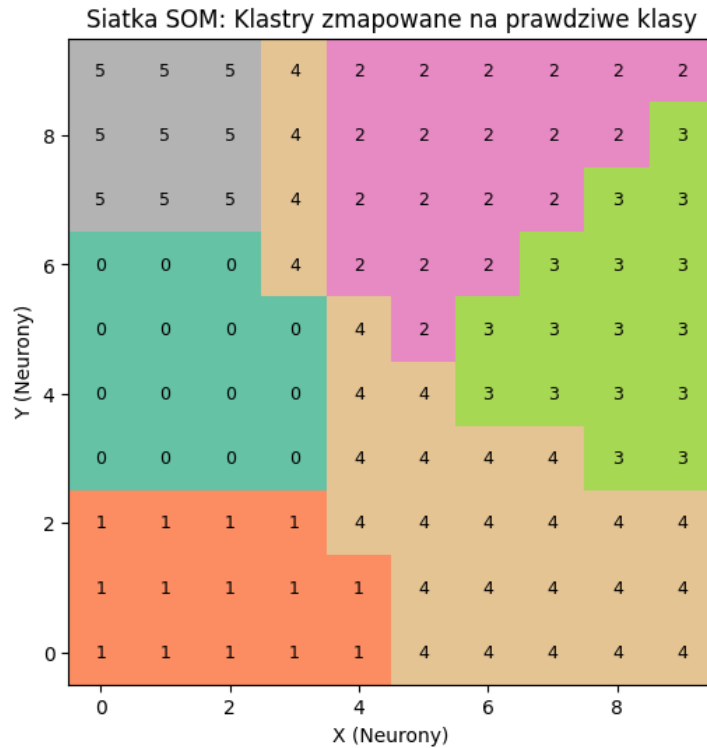
Rysunek 3: Wykres osypiska oraz wartości *Silhouette Score* dla różnych wartości klastrow w algorytmie KMeans na wagach sieci.

Następnie dokonano analizy "czystości" poszczególnych neuronów. Na 100 użytych neuronów, przypisano je do najbliższych obiektów z danych treningowych, uzyskując następujące statystyki:

- **Neurony czyste** (najbliższe obiekty pochodziły wyłącznie z jednej klasy): 81
- **Neurony mieszane** (najbliższe obiekty reprezentowały więcej niż jedną klasę): 8
- **Neurony puste** (żadna obserwacja nie była do nich przypisana jako najbliższa): 10

Poniższe zestawienie prezentuje przykładowe licznosci przypisanych obiektów dla pięciu losowo wybranych neuronów, wraz z ich współrzędnymi na siatce oraz klasą, do której należały obiekty:

- Współrzędne (2, 1): {Klasa 1: 14 obiektów}
- Współrzędne (0, 5): {Klasa 0: 9 obiektów}
- Współrzędne (4, 7): {Klasa 2: 7 obiektów, Klasa 1: 1 obiekt}
- Współrzędne (0, 1): {Klasa 1: 11 obiektów}
- Współrzędne (4, 8): {Klasa 2: 11 obiektów}



Rysunek 4: Siatka SOM po podziale na 6 klastrów.

3.2 Zbiór danych Cube

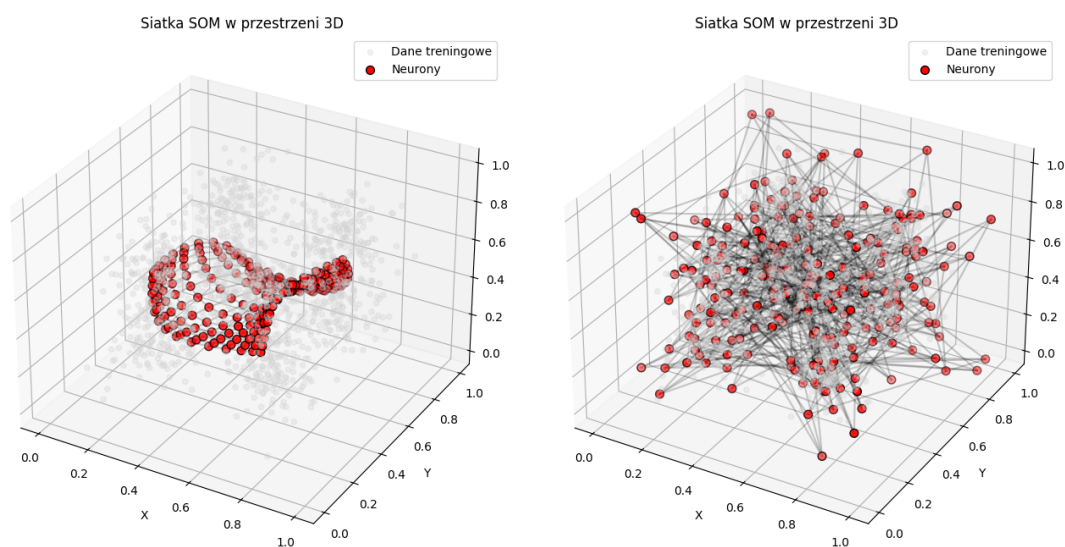
Kolejny etap eksperymentów obejmował analizę trójwymiarowego zbioru danych o strukturze sześcianu, z ośmioma wyraźnymi klastrami zlokalizowanymi w jego wierzchołkach. Ze względu na większą złożoność tego zbioru oraz dodatkowy wymiar, zastosowano powiększoną, kwadratową siatkę neuronów o wymiarach 15x15. Proces uczenia, podobnie jak w przypadku prostej płaszczyzny, trwał 100 epok.

W tym eksperymencie zbadano zachowanie dwóch różnych funkcji sąsiedztwa: gaussowskiej oraz tzw. meksykańskiego kapelusza, implementując je dla obu wariantów siatek (prostokątnej i sześciokątnej). Warto podkreślić, że o ile funkcja Gaussa dawała satysfakcjonujące rezultaty, o tyle zastosowanie "meksykańskiego kapelusza" było źródłem problemów. Przyczyną tego stanu rzeczy jest fakt, że funkcja "Meksykańskiego kapelusza" generuje znacząco większe odchylenia od klasycznej funkcji Gaussa. Wartości mogą przyjmować postać zarówno bardzo dużych liczb dodatnich jak i ujemnych, co przy aktualizacji wag ($w_{new} = w_{old} + \alpha \cdot influence \cdot (x - w_{old})$) często powodowało gwałtowne

"skoki" modelu i ostatecznie jego zniekształcenie lub - jak miało to miejsce w tym przypadku - przepełnienie na skutek wykroczenia wartości poza dopuszczalne ramy dla typu float w module numPy, dlatego funkcja ta okazała się złym wyborem.

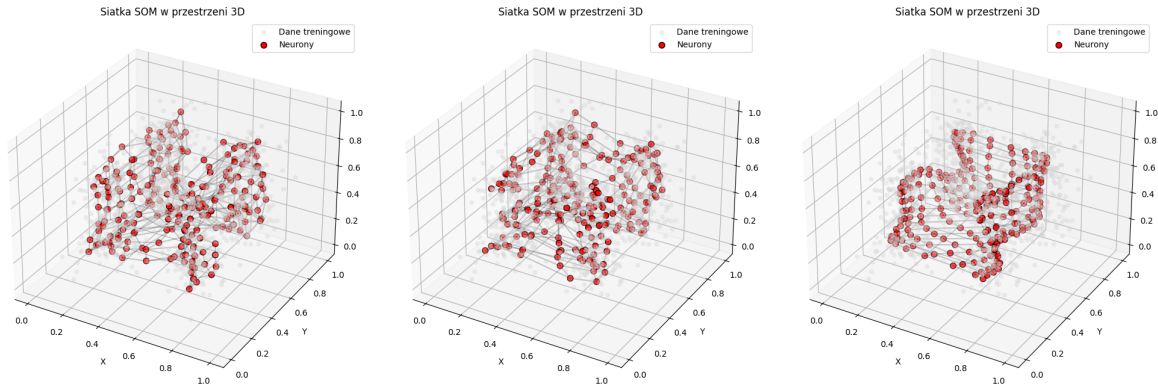
Podobnie jak dla zbioru Hexagon, przetestowano różne wartości początkowej szerokości sąsiedztwa (0.2, 0.8, 1, 2, 5). Wizualizacje wyuczonej siatki w przestrzeni 3D pokazują, że algorytm musi zniekształcać i fałdować siatkę, aby dopasować płaską topologię 2D do trójwymiarowego problemu. Skrajne wartości przyniosły następujące efekty:

- **Szerokość sąsiedztwa 5:** Skutkowała wyraźnym *underfittingiem*. Mapa nie była w stanie odpowiednio rozciągnąć się na wszystkie 8 wierzchołków sześcianu, przez co większość neuronów skupiła się w jego centrum.
- **Szerokość sąsiedztwa 0.2:** Spowodowała *overfitting*, objawiający się bardzo poszarpaną, niestabilną topologią siatki, która próbowała dopasować się do każdego lokalnego punktu, tracąc przy tym swoje właściwości uogólniające.



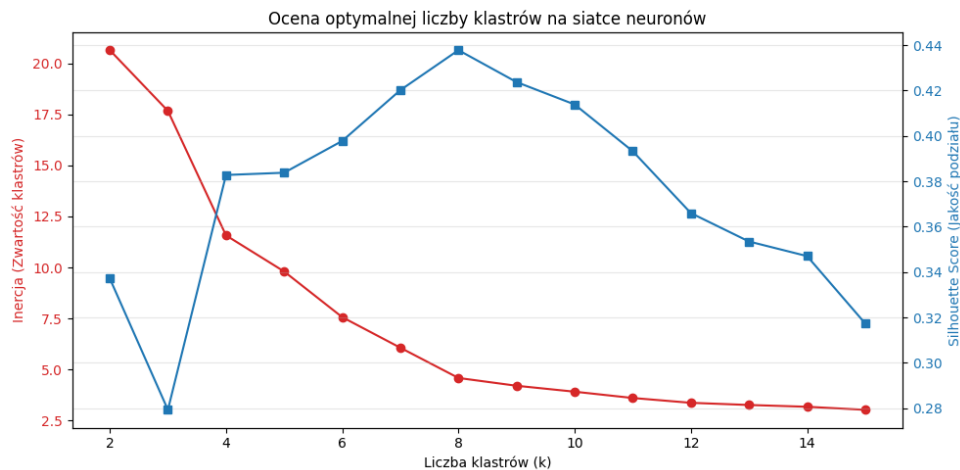
Rysunek 5: Wpływ skrajnych szerokości sąsiedztwa na siatkę w 3D: underfitting dla wartości 5 (po lewej) i overfitting dla 0.2 (po prawej).

Wartości pośrednie (0.8, 1 oraz 2) dały zauważalnie lepsze i bardziej spójne rezultaty przestrzenne. Algorytm w tych przypadkach poprawnie "rozpiął" siatkę pomiędzy wierzchołkami sześcianu. Ostatecznie to domyślna wartość 1 zapewniła optymalny kompromis pomiędzy elastycznością a zachowaniem globalnej struktury.



Rysunek 6: Poprawne dopasowanie siatki SOM dla pośrednich wartości szerokości sąsiedztwa. Od lewej: 0.8, 1 oraz 2.

W celu weryfikacji optymalnego podziału, na wagach sieci wyuczonej z parametrem równym 1 zastosowano algorytm KMeans. Wykres wartości *Silhouette Score* oraz inercji potwierdził strukturę zbioru.



Rysunek 7: Wykres osypiska oraz wartości *Silhouette Score* dla różnych wartości klastrów w algorytmie KMeans na wagach sieci (zbiór Cube).

Wyznaczono również statystyki "czystości" neuronów. Na całkowitą liczbę 225 neuronów (siatka 15x15) otrzymano bardzo zadowalające rezultaty. Poniżej zestawiono ogólne wyniki oraz przykładowe przypisania obiektów do współrzędnych (x, y) na płaskiej reprezentacji siatki:

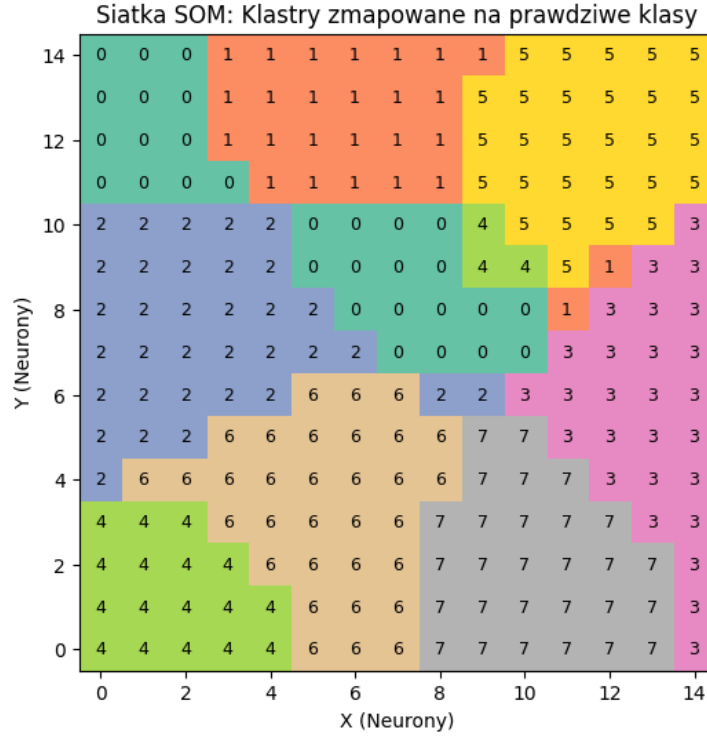
- **Liczba neuronów 'czystych'** (najbliższe obiekty tylko z 1 klasy): 182
- **Liczba neuronów 'mieszanych'** (najbliższe obiekty z >1 klas): 35

Przykładowe licznosci w neuronach:

- Współrzędne (6, 9): {Klasa 0: 7 obiektów}
- Współrzędne (0, 12): {Klasa 0: 10 obiektów}
- Współrzędne (0, 11): {Klasa 0: 5 obiektów}

- Współrzędne (5, 10): {Klasa 0: 3 obiekty}
- Współrzędne (1, 12): {Klasa 0: 7 obiektów}

Ostateczny rozkład klas i zgrupowanie obiektów na wyuczonej płaskiej mapie topologicznej zaprezentowano na poniższej wizualizacji. Można zauważyć, że pomimo skomplikowanej struktury 3D, siatka w większości dobrze odseparowała od siebie poszczególne obszary w przestrzeni 2D.



Rysunek 8: Wizualizacja płaskiej siatki SOM po przypisaniu do klastrow zmapowanych na prawdziwe klasy dla zbioru Cube.

4 Eksperymenty na rzeczywistych zbiorach danych

4.1 Zbiór obrazów MNIST

W kolejnym etapie prac algorytm przetestowano na klasycznym, rzeczywistym problemie rozpoznawania odręcznie pisanych cyfr (zbiór MNIST). W celu zmniejszenia złożoności obliczeniowej oraz czasu trwania eksperymentu, z oryginalnego zbioru wyodrębniono podzbiór liczący 2000 obserwacji. Zastosowano powiększoną siatkę neuronów o rozmiarze 15x15. Proces uczenia trwał 100 epok, a początkowa szerokość sąsiedztwa została ustawiona na wartość 1. W ramach testów sprawdzono dwie funkcje sąsiedztwa (gaussowską oraz meksykański kapelusz) dla obydwu dostępnych topologii siatki: sześciokątnej (hexagonalnej) oraz prostokątnej.

Jakość grupowania ewaluowano m.in. za pomocą metryki czystości (ang. *Purity*), która określa, w jakim stopniu klastry zawierają obiekty należące do jednej, dominującej klasy. Wzór na czystość klastrowania definiuje się następująco:

$$Purity = \frac{1}{N} \sum_{i \in \Omega} \max_{c \in C} |\omega_i \cap c| \quad (1)$$

gdzie N to całkowita liczba obserwacji, $\Omega = \{\omega_1, \omega_2, \dots, \omega_k\}$ to zbiór klastrów (w tym przypadku odpowiadający poszczególnym neuronom), a $C = \{c_1, c_2, \dots, c_j\}$ to zbiór prawdziwych klas.

Wyniki dla siatki sześciokątnej (Hexagon)

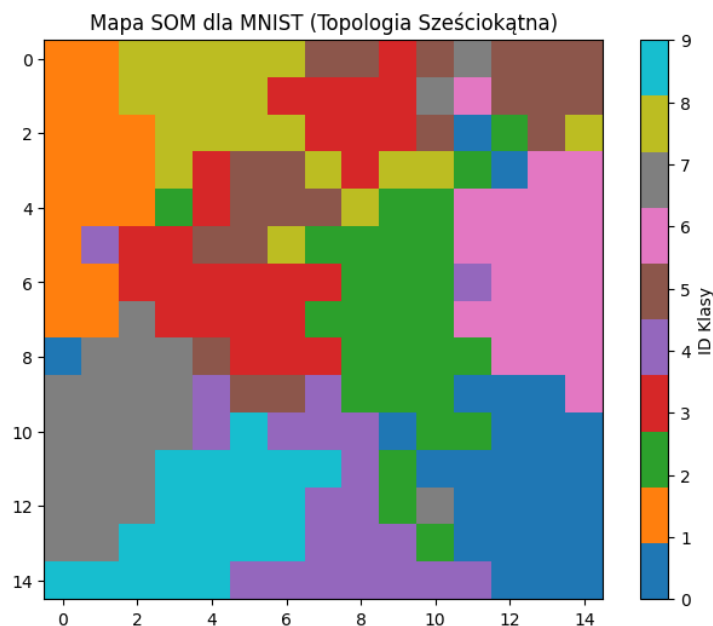
Dla siatki o topologii sześciokątnej uzyskano wartość *Purity* równą 0.86. Na podstawie wygenerowanej mapy klastrów zauważono, że sieć bardzo dobrze radzi sobie z rozpoznawaniem cyfr takich jak 0 oraz 1, rzadko myląc je z innymi klasami. Największe problemy i nałożenia obserwowano dla cyfr 3 i 8, które ze względu na duże podobieństwo wizualne często wpadały do tych samych neuronów.

Warto zauważyć, że dla bardziej skomplikowanych, rzeczywistych danych statystyki czystości poszczególnych neuronów uległy pogorszeniu w stosunku do zbiorów syntetycznych:

- **Liczba neuronów 'czystych'** (najbliższe obiekty pochodziły z 1 klasy): 100
- **Liczba neuronów 'mieszanych'** (najbliższe obiekty z >1 klas): 123

Poniżej przedstawiono przykładowe przypisania obserwacji dla losowych neuronów (współrzędne x, y):

- Współrzędne (5, 4): {Klasa 5: 9 obiektów, Klasa 6: 1 obiekt}
- Współrzędne (12, 12): {Klasa 0: 11 obiektów}
- Współrzędne (7, 9): {Klasa 4: 4 obiekty}
- Współrzędne (0, 0): {Klasa 1: 22 obiekty, Klasa 7: 1 obiekt, Klasa 4: 1 obiekt}
- Współrzędne (4, 12): {Klasa 9: 7 obiektów, Klasa 4: 1 obiekt, Klasa 7: 1 obiekt}



Rysunek 9: Siatka SOM po podziale na klastry dla zbioru MNIST (topologia sześciokątna).

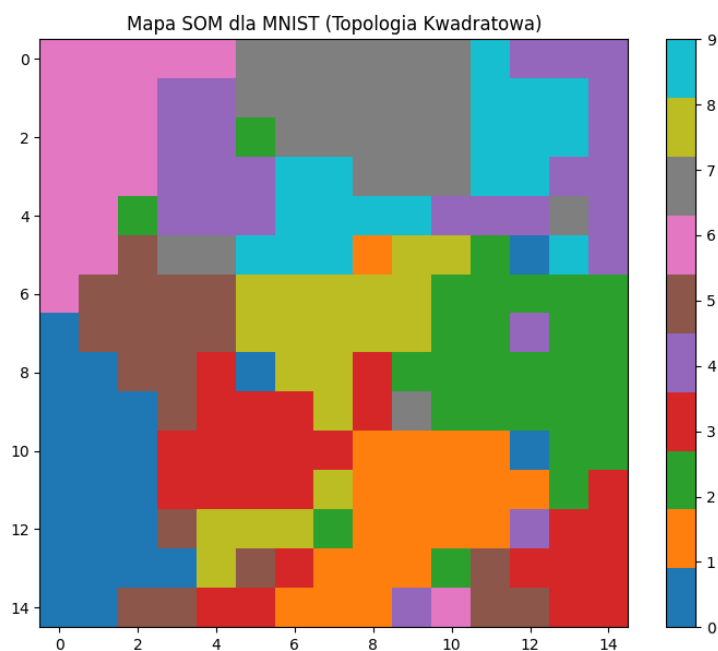
Wyniki dla siatki prostokątnej (Rectangular)

Eksperyment przeprowadzony na siatce prostokątnej przyniósł bardzo zbliżone wnioski wizualne i jakościowe, osiągając identyczną wartość *Purity* na poziomie 0.86. Mimo tego samego wyniku globalnego, rozkład obserwacji na poszczególne neurony nieznacznie się różnił – zaobserwowano więcej neuronów "czystych" kosztem mniejszego zagęszczenia na obrzeżach klastrów:

- Liczba neuronów 'czystych': 115
- Liczba neuronów 'mieszanych': 108

Przykładowe przypisania dla neuronów z siatki prostokątnej (współrzędne x, y):

- Współrzędne (11, 14): {Klasa 5: 13 obiektów, Klasa 8: 1 obiekt}
- Współrzędne (1, 11): {Klasa 0: 4 obiekty}
- Współrzędne (13, 4): {Klasa 4: 3 obiekty, Klasa 7: 4 obiekty, Klasa 9: 1 obiekt, Klasa 5: 2 obiekty}
- Współrzędne (11, 11): {Klasa 1: 19 obiektów}
- Współrzędne (9, 4): {Klasa 9: 14 obiektów, Klasa 7: 1 obiekt}



Rysunek 10: Siatka SOM po podziale na klastry dla zbioru MNIST (topologia prostokątna).

4.2 Zbiór danych HAR (Human Activity Recognition)

Ostatnim etapem badań była analiza zbioru danych HAR (Human Activity Recognition), zawierającego odczyty z czujników smartfonów, służące do klasyfikacji sześciu rodzajów aktywności fizycznej użytkownika. Podobnie jak w przypadku zbioru MNIST, wykorzystano siatkę o wymiarach 15x15, proces uczenia trwał 100 epok, a początkowy promień sąsiedztwa wynosił 1.

4.2.1 Wyniki dla siatki sześciokątnej (Hexagon)

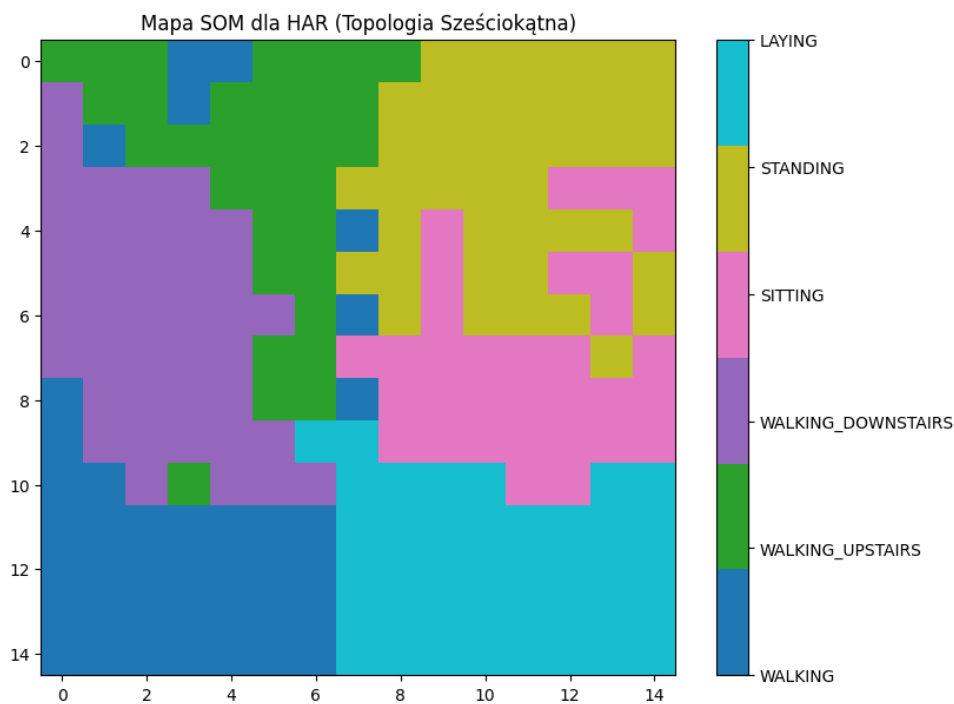
Dla siatki o topologii sześciokątnej model osiągnął czystość purity równą 0.89. Wizualizacja przypisań do klastrów (Rysunek 11) pokazuje, że sieć Kohonena najlepiej radzi sobie z wyodrębnieniem aktywności leżenia LAYING, tworząc dla niej spójny, wyraźnie oddzielony obszar na mapie. Klasy takie jak chodzenie czy wchodzenie po schodach również zajmują swoje strefy, choć z mniejszą czystością na granicach. W oczy rzuca się znaczne wymieszanie klas stania STANDING oraz siedzenia SITTING.

Statystyki dla poszczególnych neuronów dla topologii sześciokątnej przedstawiają się następująco:

- Liczba neuronów 'czystych': 105
- Liczba neuronów 'mieszanych': 116

Poniżej przedstawiono przykładowe przypisania obserwacji dla losowych neuronów (współrzędne x, y):

- Współrzędne (11, 0): {STANDING: 33, SITTING: 5}
- Współrzędne (12, 6): {STANDING: 15, SITTING: 12}
- Współrzędne (12, 1): {STANDING: 42, SITTING: 4}
- Współrzędne (12, 7): {STANDING: 24, SITTING: 49}
- Współrzędne (13, 6): {STANDING: 22, SITTING: 24}



Rysunek 11: Mapa zmapowanych klas siatki SOM dla podzbioru HAR na siatce sześciokątnej.

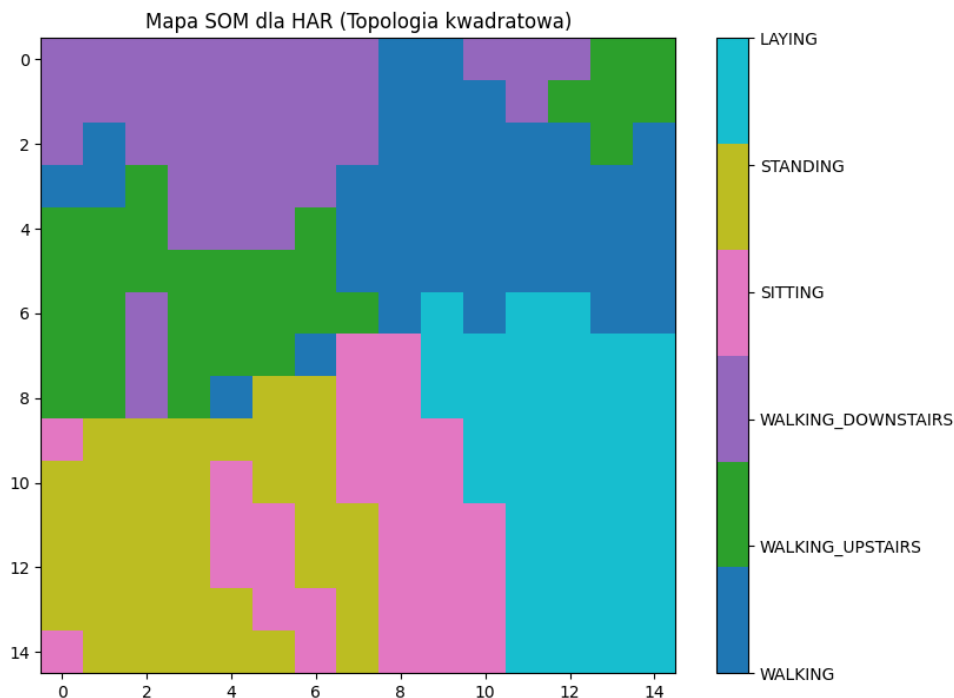
4.2.2 Wyniki dla siatki prostokątnej (Rectangular)

Podobnie jak dla topologii sześciokątnej, model wyuczony na siatce prostokątnej uzyskał purity równe 0.89. Wygenerowana mapa (Rysunek 12) ponownie uwidacznia silne przemieszanie klas *STANDING* i *SITTING*, podczas gdy klasa LAYING pozostaje dobrze odseparowana.

- Liczba neuronów 'czystych': 105
- Liczba neuronów 'mieszanych': 115

Przykładowe przypisania obserwacji do neuronów (współrzędne x, y) dla siatki prostokątnej, potwierdzające trudność w odróżnieniu stania od siedzenia:

- Współrzędne (0, 12): {STANDING: 50, SITTING: 8}
- Współrzędne (7, 12): {STANDING: 25, SITTING: 16}
- Współrzędne (3, 13): {STANDING: 26, SITTING: 3}
- Współrzędne (7, 13): {STANDING: 37, SITTING: 25}
- Współrzędne (2, 13): {STANDING: 54, SITTING: 6}



Rysunek 12: Mapa zmapowanych klas siatki SOM dla podzbioru HAR na siatce prostokątnej.